Report for the Midterm Project on Algorithms in Big Data Analysis

Yuhan Ye

March 2024

1 ℓ_1 Minimization Problem

Consider the ℓ_1 minimization problem with the infinity norm:

$$\min_x \mu\|x\|_1 + \|Ax - b\|_\infty \tag{1}$$

where $A \in \mathbb{R}^{m\times n}$ and $b \in \mathbb{R}^m$.

We solve the above problem using several methods and compare their performance.

Complete Numerical Results

Table 1: Comparison of all results with random seed 50

Solver	Residual	Norm-1	CPU Time	Err-to-cvx-mosek
cvx-mosek	4.48e-05	6.62e+01	-	-
cvx-gurobi	9.86e-05	6.62e+01	-	-
ALM_{primal}	2.50e-07	6.62e+01	0.38	1.92e-05
$ADMM_{primal}$	2.21e-07	6.62e+01	0.47	1.92e-05

Table 2: Comparison of all results with random seed 150

Solver	Residual	Norm-1	CPU Time	Err-to-cvx-mosek
cvx-mosek	2.79e-06	8.77e+01	-	-
cvx-gurobi	5.58e-08	8.77e+01	-	-
ALM_{primal}	2.37e-07	8.77e+01	0.38	1.46e-05
$ADMM_{primal}$	2.33e-07	8.77e+01	0.45	1.46e-05

1.1 Solving with CVX via Mosek and Gurobi

The original problem can be solved in CVX by calling Mosek or Gurobi. The numerical results are shown in Table 1.

Table 3: Comparison of all results with random seed 1000

Solver	Residual	Norm-1	CPU Time	Err-to-cvx-mosek
cvx-mosek	1.71e-05	7.19e+01	-	-
cvx-gurobi	4.27e-05	7.19e+01	-	-
ALM_{primal}	2.16e-07	7.19e+01	0.49	1.42e-05
$ADMM_{primal}$	1.95e-07	7.19e+01	0.45	1.42e-05

Table 4: Comparison of all results with random seed 2024

Solver	Residual	Norm-1	CPU Time	Err-to-cvx-mosek
cvx-mosek	1.63e-06	7.91e+01	-	-
cvx-gurobi	2.16e-06	7.91e+01	-	-
ALM	2.23e-07	7.91e+01	0.48	3.66e-10
ADMM	2.05e-07	7.91e+01	0.47	3.65e-10

1.2 Augmented Lagrangian Method

1.2.1 Derivation of the Iterative Scheme

First, introduce the variable $z = Ax - b$ and rewrite the original problem as

$$\begin{aligned} \min_{x,z} \quad & \mu\|x\|_1 + \|z\|_\infty, \\ \text{subject to} \quad & Ax - z = b. \end{aligned}$$

Construct the augmented Lagrangian function

$$L(x, z, u) = \mu\|x\|_1 + \|z\|_\infty + u'(Ax - z - b) + \frac{\sigma}{2}\|Ax - z - b\|_2^2.$$

The outer loop of the algorithm is as follows:

- Solve the ALM subproblem:

$$x^{k+1}, z^{k+1} = \arg\min_{x,z} \left\{ \mu\|x\|_1 + \|z\|_\infty + (\lambda^k)'(Ax - z - b) + \frac{\sigma^k}{2}\|Ax - z - b\|_2^2 \right\}, \tag{2}$$

- Update the multiplier:

$$\lambda^{k+1} = \lambda^k + \sigma(Ax^{k+1} - z^{k+1} - b),$$

- Update the penalty parameter:

$$\sigma^{k+1} = \rho\sigma^k.$$

Since the solution of (2) cannot be written in closed form, we use the proximal gradient method to solve the subproblem iteratively as the inner loop:

- The subproblem with respect to x:

Take the gradient with respect to x:

$$\nabla L_x(x) = \sigma A'(Ax - z - b) + \lambda.$$

The proximal operator of the ℓ_1 norm is

$$\text{prox}_{\mu\|\cdot\|_1}(v) = \text{sign}(v) \cdot \max(0, |v| - \mu),$$

and therefore the full iteration is

$$\begin{aligned} x_{\text{new}} &= \text{prox}_{\tau\mu\|\cdot\|_1}\left(x - \tau\nabla L_x(x)\right) \\ &= \text{sign}\left(x - \tau(\sigma A'(Ax - z - b) + \lambda)\right) \cdot \max\left(0, \left|x - \tau(\sigma A'(Ax - z - b) + \lambda)\right| - \tau\mu\right), \end{aligned}$$

where τ is the step size.

- The subproblem with respect to z:

  Take the gradient with respect to z:

$$\nabla L_z(z) = \sigma(-Ax + b + z) - \lambda.$$

  The proximal operator of the infinity norm is

$$\text{prox}_{\tau\|\cdot\|_\infty}(z) = z - \tau\text{Proj}_{\{\|\cdot\|_1 \le 1\}}\left(\frac{z}{\tau}\right),$$

  and the full iteration is

$$\begin{aligned} z_{\text{new}} &= \text{prox}_{\tau\|\cdot\|_\infty}\left(z - \tau\nabla L_z(z)\right) \\ &= z - \tau\left(\sigma(-Ax + b + z) - \lambda\right) \\ &\quad - \tau\text{Proj}_{\{\|\cdot\|_1 \le 1\}}\left(\frac{z}{\tau} - (\sigma(-Ax + b + z) - \lambda)\right), \end{aligned}$$

  where τ is the step size, which is set to $\tau = 0.1/\sigma$ in the actual computation.

1.2.2 Implementation Details

1. **Warm-start acceleration strategy.** We use a decreasing sequence of μ. Initially, we set $\mu^{(0)} = 2^{16}\mu$ and perform at most 500 outer iterations. Then we let $\mu^{(k)} = \mu^{(k-1)}/4$ and again perform at most 500 outer iterations. This process continues until $\mu^{(k)} = \mu$.

2. **Thresholding and rounding on sparse entries.** To fully exploit sparsity, after each outer iteration, if x_{ij} is sufficiently small, we regard this entry as zero and set it directly to 0. Specifically, when $\mu^{(k)} \neq \mu$, all entries smaller than $1e{-}3$ are set to 0; when $\mu^{(k)} = \mu$, all entries smaller than $1e{-}5$ are set to 0. Larger errors are allowed in the early stage to improve efficiency, while later iterations emphasize accuracy.

3. **Choice of other parameters.** Experience shows that the ALM algorithm often achieves its best performance when the number of inner iterations is very small. Therefore, we set the maximum number of inner iterations $iter_{in}$ to 1 or 2, the maximum number of outer iterations to 4000, the penalty growth factor to $\rho = 2$, and the upper bound on the penalty parameter to $1e3$. For the proximal gradient step, we use the fixed step size $\tau = 0.1/\sigma$, and the minimum decrease in the inner loop is set to $tol_{in} = 1e{-}10$.

4. **Stopping criterion.** In practice, we only check primal feasibility as the stopping criterion, namely, we stop when $\|Ax - z - b\|_2 < 1e{-}7$. Numerical tests show that the resulting accuracy is already much higher than that of the built-in solvers, while the runtime remains within half a second (see Table 1).

1.3 Solving the Original Problem by ADMM

1.3.1 Derivation of the Iterative Scheme

For the reformulated problem

$$\begin{aligned} \min_{x,z} \quad & \mu\|x\|_1 + \|z\|_\infty, \\ \text{subject to} \quad & Ax - z = b, \end{aligned}$$

construct the augmented Lagrangian function

$$L(x, z, u) = \mu\|x\|_1 + \|z\|_\infty + u'(Ax - z - b) + \frac{\sigma}{2}\|Ax - z - b\|_2^2.$$

The outer loop of the algorithm is as follows:

- Alternately solve the ADMM subproblems:

$$x^{k+1} = \arg\min_x \left\{ \mu\|x\|_1 + (\lambda^k)'(Ax - z^k - b) + \frac{\sigma^k}{2}\|Ax - z^k - b\|_2^2 \right\}, \tag{3}$$

$$z^{k+1} = \arg\min_z \left\{ \|z\|_\infty + (\lambda^k)'(Ax^{k+1} - z - b) + \frac{\sigma^k}{2}\|Ax^{k+1} - z - b\|_2^2 \right\}. \tag{4}$$

- Update the multiplier:

$$\lambda^{k+1} = \lambda^k + \tau_0\sigma(Ax^{k+1} - z^{k+1} - b), \tag{5}$$

  where τ_0 is the step size.

Similar to the previous section, we use the proximal gradient method to solve (3) as the first inner loop. The iteration is given by

$$\begin{aligned} x_{\text{new}} &= \text{prox}_{\tau\mu\|\cdot\|_1}\left(x - \tau\nabla L_x(x)\right) \\ &= \text{sign}\left(x - \tau(\sigma A'(Ax - z - b) + \lambda)\right) \cdot \max\left(0, \left|x - \tau(\sigma A'(Ax - z - b) + \lambda)\right| - \tau\mu\right). \end{aligned}$$

We also use the proximal gradient method to solve (4) as the second inner loop, with iteration

$$\begin{aligned} z_{\text{new}} &= \operatorname{prox}_{\|\cdot\|_\infty}\left(z - \tau \nabla L_z(z)\right) \\ &= z - \tau\left(\sigma(-Ax + b + z) - \lambda\right) \\ &\quad - \operatorname{Proj}_{\{\|\cdot\|_1 \le 1\}}\left(z - \tau\left(\sigma(-Ax + b + z) - \lambda\right)\right). \end{aligned}$$

1.3.2 Implementation Details

1. **Warm-start acceleration strategy and thresholding on sparse entries.** These are the same as those used in the ALM method above.

2. **Adaptive over-relaxation technique.** In (4) and (5), Ax^{k+1} is replaced by $\alpha_k Ax^{k+1} - (1-\alpha_k)(-z-b)$. We use over-relaxation for the first 3000 iterations with $\alpha_k = 1.7$; in later iterations, we set $\alpha_k = 1$.

3. **Choice of other parameters.** The maximum number of outer iterations is set to 4000, the maximum number of inner iterations $iter_{in}$ is set to 1 or 2, and the upper bound of the penalty parameter is $1e3$. For the proximal gradient step, we use the fixed step size $\tau = 0.1/\sigma$; for the multiplier update in the outer loop, we set $\tau_0 = 0.1$; and the minimum decrease in the inner loop is set to $tol_{in} = 1e{-}10$.

4. **Stopping criterion.** In practice, we only check primal feasibility as the stopping criterion, namely, we stop when $\|Ax - z - b\|_2 < 1e{-}7$. Numerical tests show that the resulting accuracy is already much higher than that of the built-in solvers, while the runtime remains within half a second (see Table 1).

1.4 Analysis of Numerical Results

The numerical results are shown in Tables 1, 2, and 3. The maximum number of outer iterations for both ALM and ADMM is set to 4000. We can see that both methods achieve significantly higher accuracy than the built-in solvers, while maintaining short runtimes.

Since our ALM formulation already applies variable splitting to the original problem, and the augmented Lagrangian subproblem is also solved by applying proximal gradient iterations separately to x and z, the final iterative scheme is already very close to ADMM. From the results, we can observe that ADMM achieves slightly higher accuracy than ALM, mainly because ADMM employs an over-relaxation strategy that accelerates convergence. Their runtimes are very similar.

2 Low-Rank Matrix Recovery

Consider the problem

$$\min_{X \in \mathbb{R}^{m\times n}} \mu\|X\|_* + \sum_{(i,j)\in\Omega} (X_{ij} - M_{ij})^2,$$

where the nuclear norm is defined by

$$\|X\|_* = \sum_i \sigma_i(X).$$

We solve the above problem using the proximal gradient method and the alternating direction method of multipliers (ADMM), respectively, and compare the performance of different methods.

Complete Numerical Results

In the following tables, we present the numerical results of the proximal gradient method and ADMM for different values of μ, m, and n. The columns on the right correspond to the optimal objective value, the number of iterations, the Frobenius norm error, and the relative error, respectively. The Frobenius norm error is defined as the Frobenius norm of the difference between the numerical solution matrix X and the original matrix A, denoted by $\|X - A\|_F$. The relative error is defined as the Frobenius norm error divided by the Frobenius norm of the original matrix A, namely $\frac{\|X-A\|_F}{\|A\|_F}$.

$m = n$	μ	Optimal Value	Number of Iterations	Frobenius Norm Error	Relative Error
40	$1e-01$	11.925711	434	11.742072	0.164300
40	$1e-02$	1.199013	458	11.498050	0.160885
40	$1e-03$	0.120194	515	14.837900	0.207618
100	$1e-01$	19.935031	364	0.316813	0.002243
100	$1e-02$	1.995340	455	0.036481	0.000258
100	$1e-03$	0.199582	412	0.058491	0.000414
200	$1e-01$	205.165989	352	0.585742	0.000892
200	$1e-02$	20.523350	432	0.101555	0.000155
200	$1e-03$	2.052371	418	0.012436	0.000019
500	$1e-01$	505.483682	459	3.771889	0.002350
500	$1e-02$	50.576710	406	2.066335	0.001287
500	$1e-03$	5.053627	429	0.877284	0.000547

Table 5: Proximal gradient method for different values of m, n, μ.

$m = n$	μ	Optimal Value	Number of Iterations	Frobenius Norm Error	Relative Error
100	$1e-01$	19.933767	491	0.338132	0.002394
100	$1e-02$	1.995225	495	0.035812	0.000254
100	$1e-03$	0.199546	444	0.018702	0.000132
200	$1e-01$	205.158461	479	0.617077	0.000940
200	$1e-02$	20.522996	451	0.112888	0.000172
200	$1e-03$	2.052369	430	0.025966	0.000040
500	$1e-01$	505.170642	524	3.429572	0.002137
500	$1e-02$	50.532383	518	2.024094	0.001261
500	$1e-03$	5.053941	419	2.218654	0.001382

Table 6: ADMM for different values of m, n, μ.

2.1 Solving by the Proximal Gradient Method

2.1.1 Derivation of the Iterative Scheme

Let

$$f(X) = \sum_{(i,j)\in\Omega} (X_{ij} - M_{ij})^2, \quad h(X) = \mu\|X\|_*,$$

and define the matrix $P \in \mathbb{R}^{m\times n}$ by

$$P_{ij} = \begin{cases} 1, & (i,j) \in \Omega, \\ 0, & \text{otherwise}, \end{cases}$$

then

$$\begin{aligned} f(X) &= \|P \odot (X - M)\|_F^2, \\ \nabla f(X) &= 2P \odot (X - M), \end{aligned}$$

where $\odot$ denotes the Hadamard product.

If we take the singular value decomposition of Y as $Y = U\mathrm{Diag}(\sigma)V^T$, then

$$\begin{aligned} \mathrm{prox}_{t_k h}(Y) &= \arg\min_Z t_k h(Z) + \frac{1}{2}\|Y - Z\|_F^2 && (6) \\ &= \arg\min_Z t_k \mu\|Z\|_* + \frac{1}{2}\|Y - Z\|_F^2 && (7) \\ &= \arg\min_Z t_k \mu\|U^T Z V\|_* + \frac{1}{2}\|U^T Y V - \mathrm{Diag}(\sigma)\|_F^2 && (8) \end{aligned}$$

Similar to the derivation of ADMM in Section 2.2, we have

$$\mathrm{prox}_{t_k h}(Y) = U\mathrm{Diag}(\max\{\sigma - t_k\mu, 0\})V^T.$$

Therefore, the proximal gradient iteration takes the form

$$\begin{aligned} Y^k &= X^k - 2t_k P \odot (X^k - M), \\ X^{k+1} &= \mathrm{prox}_{t_k h}(Y^k). \end{aligned}$$

2.1.2 Implementation Details

1. **Acceleration.** In actual runs, we found that the convergence was too slow and several tens of thousands of iterations were needed before convergence. Therefore, we adopt Nesterov acceleration, with iteration

$$\begin{aligned} Y^k &= X^{k-1} + \frac{k-2}{k+1}(X^{k-1} - X^{k-2}), \\ Z^k &= Y^k - 2t_k P \odot (Y^k - M), \\ X^{k+1} &= \mathrm{prox}_{t_k h}(Z^k). \end{aligned}$$

2. **Choice of step size.** Note that

$$\nabla f(X) = 2P \odot (X - M),$$

which is Lipschitz continuous with Lipschitz constant 2. Hence it suffices to choose a fixed step size $t_k < \frac{1}{2}$. In our experiments we set $t_k = 0.3$.

3. **Warm-start acceleration strategy.** We use a decreasing sequence of μ. Initially, we set $\mu^{(0)} = 2^{16}\mu$ (if $\mu = 1e{-}1$, we use 2^{12} instead) and perform at most 100 iterations. Then we let $\mu^{(k)} = \mu^{(k-1)}/16$ and again perform at most 100 outer iterations. This continues until $\mu^{(k)} = \mu$.

4. **Stopping criterion.** We set the maximum number of iterations to 5000 and the minimum number of iterations to 400 (to ensure that μ has already decreased to its original value; when $\mu = 1e{-}1$, the minimum number of iterations is 300). The stopping criterion is

$$\frac{(f(X^{k+1}) + h(X^{k+1})) - (f(X^k) + h(X^k))}{f(X^{k+1}) + h(X^{k+1})} < 10^{-6},$$

at which point the decrease of the objective function is regarded as essentially zero.

2.2 Solving by ADMM

2.2.1 Derivation of the Iterative Scheme

Using variable splitting, let $E = M - X$. Then the problem can be rewritten as

$$\min_{X,E} \ \mu\|X\|_* + \|E\|^2_{F(\Omega)}, \ \text{s.t.} \ X + E = M$$

where

$$\|E\|_{F(\Omega)} = \sum_{(i,j)\in\Omega} E_{ij}^2.$$

The augmented Lagrangian function is

$$L(X, E, \Lambda) := \mu\|X\|_* + \|E\|^2_{F(\Omega)} + \langle \Lambda, X + E - M\rangle + \frac{1}{2\beta}\|X + E - M\|_F^2.$$

The ADMM procedure is as follows:

1. Initialization: choose $X^1, E^1, \Lambda^1, \tau, \beta$. Here τ is the iteration step size, usually chosen in $(0, \frac{1+\sqrt{5}}{2}]$.

2. Perform alternating iterations (outer loop). At the j-th iteration, execute

$$\begin{aligned} X^{j+1} &:= \arg\min_X L(X, E^j, \Lambda^j), \\ E^{j+1} &:= \arg\min_E L(X^{j+1}, E, \Lambda^j), \\ \Lambda^{j+1} &:= \Lambda^j + \frac{\tau}{\beta}(X^{j+1} + E^{j+1} - M). \end{aligned}$$

Solving the X-subproblem Note that

$$\begin{aligned} X^{j+1} &= \arg\min_X L(X, E^j, \Lambda^j) \\ &= \arg\min_X \mu\|X\|_* + \frac{1}{2\beta}\left\|X - \left(M - E^j - \beta\Lambda^j\right)\right\|_F^2. \end{aligned}$$

Let $S^j = M - E^j - \beta\Lambda^j$, and take the singular value decomposition of S^j:

$$M - E^j - \beta\Lambda^j = U\mathrm{Diag}(\sigma)V^T.$$

Note that both the nuclear norm and the Frobenius norm are invariant under multiplication by orthogonal matrices on the left and right, that is,

$$\|X\|_* = \|U^T XV\|_*, \qquad \|X\|_F = \|U^T XV\|_F.$$

Thus, by letting $Y = U^T XV$, the problem is equivalent to solving the Y-subproblem

$$X^{j+1} = \arg\min_X \mu\|Y\|_* + \frac{1}{2\beta}\|Y - \mathrm{Diag}(\sigma)\|_F^2.$$

Let

$$s(\sigma_i) = \begin{cases} \sigma_i - \mu\beta, & \text{if } \sigma_i - \mu\beta > 0, \\ 0, & \text{otherwise.} \end{cases}$$

It is easy to see that the minimum of $\mu\|Y\|_* + \frac{1}{2\beta}\|Y - \mathrm{Diag}(\sigma)\|_F^2$ is attained when Y is diagonal. We set

$$Y = \mathrm{Diag}(y_1, y_2, \ldots, y_r), \qquad (r = \mathrm{rank}(X)),$$

and it is straightforward to verify that the minimum is achieved when $y_i = s(\sigma_i)$. Therefore,

$$\begin{aligned} X^{j+1} &= UY_{\min}V^T \\ &= U\mathrm{Diag}(s(\sigma))V^T. \end{aligned}$$

Solving the E-subproblem For the subproblem

$$E^{j+1} = \arg\min_E L(X^{j+1}, E, \Lambda^j),$$

we have

$$E^{j+1} = \arg\min_E L(X^{j+1}, E, \Lambda^j) \tag{9}$$

$$= \arg\min_E \|E\|_{F(\Omega)}^2 + \frac{1}{2\beta}\left\|E - \left(M - X^{j+1} - \beta\Lambda^j\right)\right\|_F^2. \tag{10}$$

Let

$$T^j = M - X^{j+1} - \beta\Lambda^j,$$

then the minimizer with respect to E is given by

$$E_{kl} = \begin{cases} \frac{T^j_{kl}}{2\beta+1}, & \text{if } (k,l) \in \Omega, \\ T^j_{kl}, & \text{otherwise.} \end{cases}$$

2.2.2 Implementation Details

1. **Warm-start acceleration strategy.** We use a decreasing sequence of μ. Initially, we set $\mu^{(0)} = 2^{16}\mu$ and perform at most 100 iterations. Then we let $\mu^{(k)} = \mu^{(k-1)}/16$ and again perform at most 100 outer iterations. This continues until $\mu^{(k)} = \mu$.

2. **Stopping criterion.** The first stopping criterion is

$$\frac{\|X^j + E^j - M\|_F}{\|M\|_F} < 10^{-6},$$

which indicates that primal feasibility is essentially satisfied. The second stopping criterion is

$$\frac{(f(X^{k+1}) + h(X^{k+1})) - (f(X^k) + h(X^k))}{f(X^{k+1}) + h(X^{k+1})} < 10^{-6},$$

which means that the decrease in the objective function is essentially zero. The minimum number of iterations is set to 400 (so that μ has already decreased to its original value), and the maximum number of iterations is 10000.

2.3 Result Analysis

From Table 5, we can see that Nesterov acceleration significantly improves the convergence speed of the proximal gradient method, reducing the required number of iterations to only a few hundred. The method is able to recover the original matrix A quite well, and can therefore be regarded as one of the most suitable algorithms for this problem.

Combining this observation with the results in Table 6, we find that after a comparable number of iterations, ADMM achieves almost the same results as the proximal gradient method. Although its performance is still quite good, it does not show the clear advantage over the proximal gradient method that we initially expected. On the one hand, this may be because the stopping conditions used for ADMM are actually stricter, even though the stopping criteria of the two methods look similar. On the other hand, the splitting of X and E together with the alternating minimization procedure may not be especially well suited to this problem. In addition, other algorithmic parameters, such as the step size, may still be improved.

Comparing the last two columns of the tables, we find that when m, n are small (for example, 40), the solution X of the optimization problem differs significantly from the original matrix A. We conjecture that this gap is mainly due to the nonequivalence between the optimization problem and the original low-rank matrix recovery problem, namely, the optimizer X^* of the optimization problem may satisfy $X^* \neq A$. When m, n are larger (for example, 100, 200, or 500), the recovered solution X becomes very close to the original matrix A. From the experimental

results, we also observe that within a certain range, a smaller value of μ leads to a smaller gap between the numerical solution X and the original matrix A, because our warm-start acceleration strategy is better suited to smaller values of μ. In addition, choosing a smaller μ tends to preserve more information in the recovered solution X (that is, the rank tends to be higher), but if μ is too small (for example, below $1e-4$), the convergence may become too slow. Therefore, in practical applications, choosing an appropriate value of μ is crucial for obtaining the best recovery performance.